

ECM1414

Data Structures and Algorithms

Coursework | Group 35

Max Thrift-Brothers
Cameron Russell
Utku Ozturk

Table of Contents

1. Introduction	2
2. Design and Approach	2
a. Prerequisites and Dependencies	2
b. Program Structure	2
c. Data Structures	3
3. Algorithms	4
a. Brute Force	4
b. Dynamic Programming	6
4. Testing and Results	7
a. Test 1	7
b. Test 2	8
c. Test 3 - input small	8
d. Test 4 - input medium	8
e. Test 5 - input large	9
f. Graph of Results	9
g. Table of Results	10
5. References	10

1. Introduction

A university society committee is planning to host an event weekend. They have a selection of activities to put on, but don't know which subset of activities to choose. They would like to maximise the enjoyment of the society members whilst staying within their available budget.

Our project aims to analyse a list of activities that this society could undertake and determine the optimal selection of activities to host.

We have achieved this using two algorithms: a brute-force approach and a dynamic programming approach. Both approaches are timed, and their respective solutions are output in a human-readable format.

2. Design and Approach

Prerequisites and Dependencies

Our project was implemented using Python 3.13 and used several built-in libraries:

- `sys` - used to extract terminal arguments and terminate the program.
- `pathlib.Path` - used to construct the file path to the specified sample file whilst being platform independent.
- `time` - used to measure the execution time of each algorithm.
- `itertools.combinations` - used to generate all subsets of a given size.

Program Structure

We have aimed for a modular program structure which aids collaborative development and maintainability.

Each algorithm is implemented in its own Python module:

`brute_force.py` and `dynamic_method.py`. Each module is imported and used within a main program called `event_planner.py`. By separating each algorithm into its own module, it allows separate team members to work on each algorithm simultaneously.

The purpose of `event_planner.py` is to communicate with and link all three helper modules. Its main job is to receive the optimal solution calculated by each algorithm and output each in a human-readable format.

The third helper module is named `file_input.py`, and its job is to parse the input file, ensuring it is in the correct format for the two algorithms to analyse. It also ensures that the input file exists and informs the user with a detailed error message if it doesn't.

The `brute_force.py` module is split into two functions: `find_optimal_solution()` and `generate_subset()`. The `generate_subset()` function utilises the `combinations` function from the built-in module `itertools` to generate all subsets of a given size. I have chosen to extract this operation into a standalone function so that a custom subset calculator could be easily implemented in the future. This idea links back to the maintainability design aspect mentioned above. The second function, `find_optimal_solution()`, contains the logic to analyse each generated subset and return the result.

Data Structures

We chose to use a Python list to store the activities in, because we required a mutable data structure with direct access and that could store elements of different sizes.

Each element of this list is a tuple. Since each activity's data doesn't need to change once parsed from the input file and consists of different types, an immutable data structure that could store elements of different data types was necessary.

The first tuple, within this list, contains the metadata for that particular input file: number of activities, available time, and available budget. All subsequent tuples contain the data for each respective activity: activity name, required time, required cost, and enjoyment value.

Both algorithms utilise a dictionary to store the optimal solution, because it is a convenient structure to store multiple types and sizes of data. This dictionary's structure is as follows:

Key	Value
<code>subset_indices / subset</code>	The optimal activity subset.
<code>time_used</code>	The total number of hours this subset requires.
<code>budget_used</code>	The total cost of this subset. Always $\leq$ available budget.
<code>total_enjoyment_value</code>	The total enjoyment that society members will experience.

3. Algorithms

Brute Force

My implementation of the brute force approach involves systematically generating all subsets of a given size and evaluating each subset against the budget constraint, whilst ensuring this subset improves the enjoyment value. My algorithm then repeats this process for all sizes of subsets until it has exhausted all possibilities.

Once finished, my algorithm will have found the optimal selection of activities that maximises the enjoyment value, whilst adhering to the budget constraint.

To return the optimal solution, my algorithm packages up the chosen subset, time used, budget used (always $\leq$ available budget), and total enjoyment value within a dictionary. This dictionary is then returned to the `event_planner.py` module to be output in a human-readable format.

As mentioned in the Design and Approach section, the `brute_force.py` module is comprised of two functions: `find_optimal_solution()` and `generate_subset()`. The `find_optimal_solution()` function contains the logic to evaluate each subset. Below is the pseudocode for this function:

```
INTEGER available_budget <- activity_list[0][2]
INTEGER number_of_activities <- activity_list[0][0]

optimal_subset <- DICTIONARY {
  "subset_indices": ARRAY[INTEGER],
  "time_used": 0,
  "budget_used": 0,
  "total_enjoyment_value": 0
}

ARRAY[INTEGER] activity_indices OF SIZE number_of_activities

FOR INTEGER i FROM 1 TO number_of_activities DO
  activity_indices[i-1] <- i
END FOR

FOR INTEGER size_of_subset FROM 1 TO number_of_activities DO
  ARRAY[ARRAY[INTEGER]] subsets <- generate_subset(activity_indices,
size_of_subset)

  FOR ARRAY[INTEGER] subset IN subsets DO
    INTEGER sum_of_budget <- 0
    INTEGER sum_of_time <- 0
```

```

    INTEGER sum_of_enjoyment_values <- 0

    FOR INTEGER activity_index IN subset DO
        sum_of_time += activity_list[activity_index][1]
        sum_of_budget += activity_list[activity_index][2]
        sum_of_enjoyment_values += activity_list[activity_index][3]
    END FOR

    IF (sum_of_enjoyment_values >
optimal_subset['total_enjoyment_value'] AND sum_of_budget <=
available_budget) THEN
        optimal_subset["subset_indices"] <- subset
        optimal_subset["time_used"] <- sum_of_time
        optimal_subset["budget_used"] <- sum_of_budget
        optimal_subset["total_enjoyment_value"] <-
sum_of_enjoyment_values
    END IF
    END FOR
END FOR

RETURN optimal_subset

```

This function first stores the available budget and the number of activities in two variables. It then initialises a dictionary to hold the optimal solution, inserting initial values into each field. The algorithm then generates a list of indices that are used to generate the subsets. This is done so as not to duplicate the activity data multiple times. There is a main outer loop with the purpose of enabling the algorithm to analyse subsets of different sizes. Within this loop, the algorithm calls the `generate_subset()` function and then proceeds to analyse each subset generated. For each subset, the algorithm initialises three tracker variables used to calculate the properties of that particular subset. Then, the third loop analyses each activity in the current subset, updating the tracker variables. Once the subset is analysed, the algorithm checks if this subset improves the enjoyment value whilst adhering to the budget and if it does, it updates the optimal solution. This process is then repeated for all possible subsets. Once finished, the function returns the optimal selection of activities.

For n activities, there are $2^n - 1$ subsets, excluding the empty set. Therefore, the outer loop iterates $2^n - 1$ times, but the condition `size_of_subset <= number_of_activities` is checked 2^n times, the last check being the check that terminates the loop. Since the first nested loop will iterate a different number of times depending on the value of `size_of_subset`, you can use the choose function to express the number of iterations relative to the value of `size_of_subset`: $C(\text{number_of_activities}, \text{size_of_subset})$. Then, the second nested loop will always iterate `size_of_subset` times. Therefore, the overall algorithm is of exponential time complexity.

Dynamic Programming

All of dynamic programming is contained within the `dynamic_method.py` file; it utilises a 2-Dimensional Dynamic Programming table ([Geeks for Geeks, 2026](#)) as well as the 0/1 knapsack problem ([Geeks for Geeks, 2026](#)). The table keeps track of previously computed results and uses them to compute the next set of activities. This approach keeps the time complexity and cost of computation low. When the cost exceeds the budget, the program stops adding more columns and moves on.

Pseudocode for creation of table:

```
CREATE dp_table as an array

FOR i FROM 0 TO num_activities DO
    CREATE a row with length of av_budget, each with 0
    ADD a row to table
END FOR

FOR i FROM 0 TO num_activities DO
    act_cost <= cost of activity i
    act_enjoy <= enjoyment of activity i

    FOR t FROM 0 TO av_budget DO
        IF act_cost <= t THEN
            dp_table ADD MAX(dp_table[t-1][i], dp_table[t-1][i -act_cost]
+ act_enjoy to [t][i])
        ELSE
            dp_table ADD dp_table[t-1][i] to [t][i]
        END IF
    END FOR
END FOR
```

Pseudocode for data extraction of DP table:

```
CREATE LIST subsets
SET remaining_budget <= av_budget
SET total_time <= 0
SET used_budget <=0
SET total_enjoyment <= 0

FOR i FROM num_activities TO 0 DO
    act_cost <= cost of activity i
    act_enjoy <= enjoyment value of activity i
    act_time <= time for activity i

    IF budget >= act_cost and dp_table[i][budget] != dp_table[i-
1][budget] DO
        ADD activity i to subsets
    END IF
END FOR
```

```

        budget <= budget - act_cost
        total_time <= total_time - act_time
        total_enjoy <= total_enjoy + act_enjoy
        required_budget <= required_budget + act_cost
    END IF
END FOR

OUTPUT subsets, total_time, required_budget and total_enjoy

```

This algorithm first fills in the DP table with each required time and enjoyment value, taking the previous stored values into account. Where each value `db[i][b]` represents the enjoyment value of the first `i` activities. This stops the overcomputation of overlapping subproblems.

After it is full, the algorithm traverses the DP table backwards, adding the activity if the results of adding it improve the subset without going over budget.

By using a dynamic method like this, with a DP table, the overall time complexity and efficiency of solving such a problem are reduced. The table has a size of number of activities by available Budget, therefore the overall time complexity is $O(nB)$, which is better than quadratic time, but not as good as linear time. This is a big improvement from the $O(2^n)$ complexity of the brute force algorithm.

4. Testing and Results

To test our program, we will be performing several edge case scenarios (erroneous entry) and analysing the output when a correct input is given.

Tests 3 to 5 will compare the performance of the two algorithms with increasing input size.

Test 1

Test of an erroneous file name:

```

(base) maxtb@Maxs-MacBook-Air-10 Code % python event_planner.py input_small.txt
File 'input_small.txt' doesn't exist in the directory: '/Users/maxtb/Library/CloudStorage/OneDrive-UniversityofExeter/Year 1/ECM1414 - Data Struc & Algorithms/Group-35-Data-Structures-and-Algorithms-CW/Input_Files'.

```

Test passed! The program correctly informs the user that the file does not exist.

Test 2

Test to observe how the algorithm performs when no input file name is given:

```
(base) maxtb@Maxs-MacBook-Air-10 Code % python event_planner.py
Please provide a file name: e.g. 'input_small.txt'.
```

Test passed! The program correctly informs the user that they must supply a file name and provides an example format.

Test 3 - input_small

```
(base) maxtb@Maxs-MacBook-Air-10 Code % python event_planner.py input_small.txt
=====
Event Planner - Results
=====

Input File: input_small.txt
Available Time: 10 hours
Available Budget: £200

--- BRUTE FORCE ALGORITHM ---
Selected Activities:
- Campus-Tour (2 hours, £20, enjoyment 50)
- Game-Night (3 hours, £80, enjoyment 120)
- Pizza-Workshop (2 hours, £60, enjoyment 100)
- Hiking (5 hours, £30, enjoyment 140)

Total Enjoyment: 410
Total Time Used: 12 hours
Total Cost: £190

Execution Time: 0.000047 seconds

--- DYNAMIC PROGRAMMING ALGORITHM ---
Selected Activities:
- Campus-Tour (2 hours, £20, enjoyment 50)
- Game-Night (3 hours, £80, enjoyment 120)
- Pizza-Workshop (2 hours, £60, enjoyment 100)
- Hiking (5 hours, £30, enjoyment 140)

Total Enjoyment: 410
Total Time Used: 12 hours
Total Cost: £190

Execution Time: 0.000057 seconds
=====
```

For the input file of 5 activities, both algorithms produced the same result.

The Brute Force method took 4.7×10^{-5} s to compute, whereas the dynamic method took 5.7×10^{-5} s.

Here, you can see that the more complicated algorithm is less efficient with smaller input sizes.

Test 4 - input_medium

```
(base) maxtb@Maxs-MacBook-Air-10 Code % python event_planner.py input_medium.txt
=====
Event Planner - Results
=====

Input File: input_medium.txt
Available Time: 15 hours
Available Budget: £300

--- BRUTE FORCE ALGORITHM ---
Selected Activities:
- Welcome-BBQ (3 hours, £50, enjoyment 80)
- Karaoke-Night (2 hours, £40, enjoyment 70)
- Film-Screening (3 hours, £30, enjoyment 90)
- Sports-Tournament (4 hours, £60, enjoyment 110)
- Pub-Quiz (2 hours, £25, enjoyment 60)
- Cooking-Class (3 hours, £75, enjoyment 105)
- Open-Mic (2 hours, £20, enjoyment 50)

Total Enjoyment: 565
Total Time Used: 19 hours
Total Cost: £300

Execution Time: 0.001547 seconds

--- DYNAMIC PROGRAMMING ALGORITHM ---
Selected Activities:
- Welcome-BBQ (3 hours, £50, enjoyment 80)
- Karaoke-Night (2 hours, £40, enjoyment 70)
- Film-Screening (3 hours, £30, enjoyment 90)
- Sports-Tournament (4 hours, £60, enjoyment 110)
- Pub-Quiz (2 hours, £25, enjoyment 60)
- Cooking-Class (3 hours, £75, enjoyment 105)
- Open-Mic (2 hours, £20, enjoyment 50)

Total Enjoyment: 565
Total Time Used: 19 hours
Total Cost: £300

Execution Time: 0.000260 seconds
=====
```

For the input file with 12 activities, both algorithms produced the same result.

The Brute Force method took 1.5×10^{-3} s to compute, whereas the dynamic method took 2.6×10^{-4} s.

With this input size, the more complicated algorithm is more efficient.

Therefore, the dynamic algorithm is only suited for input sizes of 12 or more.

Test 5 - input_large

```
(base) maxtb@Maxs-MacBook-Air-10 Code % python event_planner.py input_large.txt
=====
Event Planner - Results
=====

Input File: input_large.txt
Available Time: 20 hours
Available Budget: £500

--- BRUTE FORCE ALGORITHM ---
Selected Activities:
- Orientation-Walk (1 hour, £10, enjoyment 30)
- Ice-Breaker-Games (2 hours, £20, enjoyment 50)
- Movie-Marathon (5 hours, £60, enjoyment 140)
- Charity-Run (3 hours, £15, enjoyment 70)
- Trivia-Night (2 hours, £30, enjoyment 65)
- Campus-Scavenger-Hunt (3 hours, £25, enjoyment 75)
- Photography-Walk (3 hours, £40, enjoyment 85)
- Poetry-Slam (2 hours, £20, enjoyment 55)
- Dance-Workshop (2 hours, £50, enjoyment 90)
- Baking-Competition (3 hours, £55, enjoyment 95)
- Board-Game-Cafe (3 hours, £45, enjoyment 80)
- Volunteering-Event (4 hours, £10, enjoyment 60)
- Park-Picnic (3 hours, £30, enjoyment 65)
- Stargazing-Trip (4 hours, £50, enjoyment 100)
- Crafts-Fair (2 hours, £40, enjoyment 75)

Total Enjoyment: 1135
Total Time Used: 42 hours
Total Cost: £500

Execution Time: 16.469223 seconds
```

For the list of 25 possible activities, the Brute Force method took 16s to compute, whereas the dynamic method took 7.9×10^{-4} s.

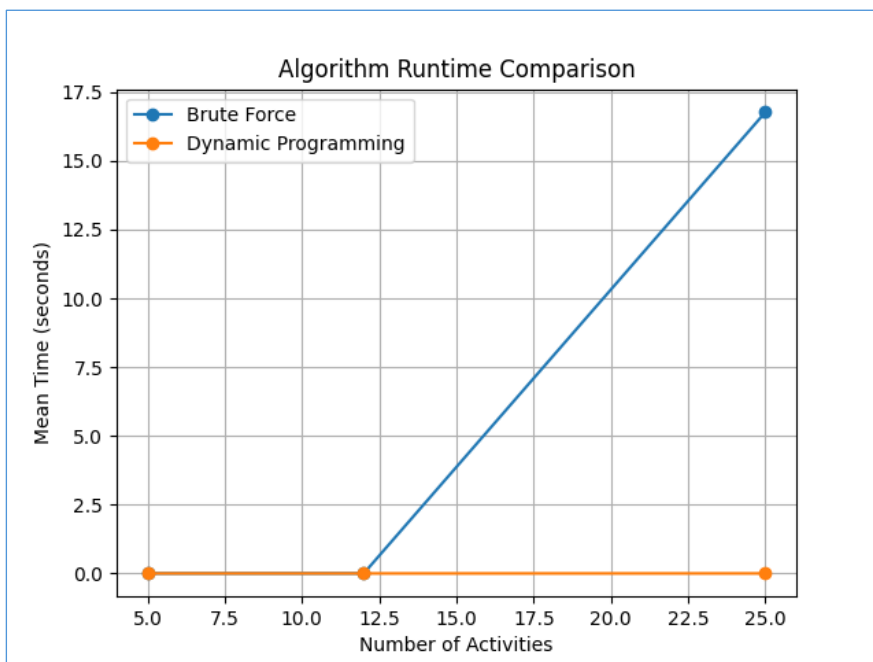
This is where a substantial difference in computation time is first seen. For files of larger size, the Brute Force Algorithm struggles or even fails. The Dynamic Programming method still manages to compute 1000 activities in less than a second.

```
--- DYNAMIC PROGRAMMING ALGORITHM ---
Selected Activities:
- Orientation-Walk (1 hour, £10, enjoyment 30)
- Ice-Breaker-Games (2 hours, £20, enjoyment 50)
- Movie-Marathon (5 hours, £60, enjoyment 140)
- Charity-Run (3 hours, £15, enjoyment 70)
- Trivia-Night (2 hours, £30, enjoyment 65)
- Campus-Scavenger-Hunt (3 hours, £25, enjoyment 75)
- Photography-Walk (3 hours, £40, enjoyment 85)
- Poetry-Slam (2 hours, £20, enjoyment 55)
- Dance-Workshop (2 hours, £50, enjoyment 90)
- Baking-Competition (3 hours, £55, enjoyment 95)
- Board-Game-Cafe (3 hours, £45, enjoyment 80)
- Volunteering-Event (4 hours, £10, enjoyment 60)
- Park-Picnic (3 hours, £30, enjoyment 65)
- Stargazing-Trip (4 hours, £50, enjoyment 100)
- Crafts-Fair (2 hours, £40, enjoyment 75)

Total Enjoyment: 1135
Total Time Used: 42 hours
Total Cost: £500

Execution Time: 0.000794 seconds
```

Graph of Results



This graph illustrates the relationship between the number of activities in a dataset and the time it took to compute. By using the average time over 10 tests of both algorithms for each of the small, medium and large files.

Table of Results

	File Name	No of Activites	Time/s										Mean Time/s
			1	2	3	4	5	6	7	8	9	10	
Brute Force	input_small	5	8.20E-05	5.80E-05	1.70E-05	5.80E-05	5.10E-05	4.80E-05	4.90E-05	5.70E-05	4.80E-05	4.90E-05	5.17E-05
	input_medium	12	1.16E-03	1.30E-03	1.43E-03	1.11E-03	1.29E-03	1.23E-03	1.63E-03	1.31E-03	1.25E-03	1.30E-03	1.30E-03
	input_large	25	1.69E+01	1.67E+01	1.68E+01	1.64E+01	1.68E+01	1.68E+01	1.69E+01	1.67E+01	1.69E+01	1.68E+01	16.7681465
Dynamic	input_small	5	5.50E-05	6.20E-05	6.10E-05	6.30E-05	5.40E-05	5.90E-05	6.00E-05	6.00E-05	5.90E-05	5.90E-05	5.92E-05
	input_medium	12	1.95E-04	2.20E-04	2.33E-04	1.93E-04	2.25E-04	2.05E-04	2.73E-04	2.18E-04	2.19E-04	2.14E-04	2.20E-04
	input_large	25	7.77E-04	7.69E-04	7.70E-04	7.62E-04	7.70E-04	7.56E-04	7.71E-04	7.71E-04	7.80E-04	8.13E-04	0.0007739

All of these tests have been performed on a 2024 MacBook Air M3 with 16GB of RAM, using the latest versions of all software in PyCharm Pro.

5. References

- Geeks for Geeks (2026, July 12). *Itertools Combinations() function - Python*. Available at: <https://www.geeksforgeeks.org/python/python-itertools-combinations-function/> (Accessed: 27 January 2026)
- Geek for Geeks (2026, January 29). *0/1 Knapsack Problem*. Available at: <https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/> (Accessed: 12 February 2026)
- Geek for Geeks (2026, January 29). *Dynamic Programming or DP*. Available at: <https://www.geeksforgeeks.org/competitive-programming/dynamic-programming/> (Accessed: 12 February 2026)